

UFUG 2602

Data Structure and Algorithm Design

Beiyan Cao, Tengfei Chang

CES and IoT Thrust

Hong Kong University of Science and Technology (Guangzhou)

Recap:

- Data Structure (Representation/Organization)+Operations = Data Type
- Data Types
 - Basic Data Type provided by a programming Language
 - User Defined Data Type by programmers when needed
- A User Defined Data Type
 - Can be built on other data types (either basic or user-defined).
- Abstract Data Types
 - Two Separations:
 - The definition of the data values is separated from their representation.
 - The definition of the operations is separated from their implementation.
 - Encapsulation: Hides internal details of implementation.

```

1 # Tic-Tac-Toe Game
2 class TicTacToe:
3     '''
4     Management of a Tic-Tac-Toe Game
5     No strategy
6     '''
7     def __init__(self):
8         ''' start a new game'''
9         self._board = [ [' ' ] * 3 for j in range(3) ]
10        self._player = 'X'
11
12    def mark(self, i, j):
13        ''' Put X or O mark at position (i,j) for next player's turn '''
14
15        if not(0 <= i <= 2 and 0 <= j <= 2):
16            raise ValueError('Invalid board position')
17        if self._board[i][j] != ' ':
18            raise ValueError('Board position occupied')
19        if self.winner() is not None:
20            raise ValueError('Game is already complete')
21        self._board[i][j] = self._player
22
23        if self._player == 'X':
24            self._player = 'O'
25        else:
26            self._player = 'X'
27
28    def _is_win(self, mark):
29        ''' Check whether the board configuration is a win for a given player'''
30        board = self._board
31
32        return (mark == board[0][0] == board[0][1] == board[0][2] or # row 0
33                mark == board[1][0] == board[1][1] == board[1][2] or # row 1
34                mark == board[2][0] == board[2][1] == board[2][2] or # row 2
35                mark == board[0][0] == board[1][0] == board[2][0] or # column 0
36                mark == board[0][1] == board[1][1] == board[2][1] or # column 1
37                mark == board[0][2] == board[1][2] == board[2][2] or # column 2
38                mark == board[0][0] == board[1][1] == board[2][2] or # diagonal
39                mark == board[0][2] == board[1][1] == board[2][0]) # rev diag
40
41    def winner(self):
42        ''' return mark of the winning player, or None to indicate a tie'''
43        for mark in 'XO':
44            if self._is_win(mark):
45                return mark
46        return None
47
48    def __str__(self):
49        ''' return the string representation of the current game board'''
50        rows = ['|'.join(self._board[r]) for r in range(3)]
51        return '\n-----\n'.join(rows)
52

```

```

53
54 game = TicTacToe()
55
56 # X moves          # O moves
57 game.mark(1,1);    game.mark(0,2)
58 game.mark(2,2);    game.mark(0,0)
59 game.mark(0,1);    game.mark(2,1)
60 game.mark(1,2);    game.mark(1,0)
61 game.mark(2,0);
62
63 print(game)
64 winner = game.winner()
65 if winner is None:
66     print('Tie')
67 else:
68     print(winner, 'wins')
69

```

```

O|X|O
-----
O|X|X
-----
X|O|X
Tie

```

In class Exercise: Generalized Win Check

- Rewrite the `_is_win(mark)` function **without hard-coded indices**, using **loops over a 2-D array**.
- The solution should:
 - Check **all rows**
 - Check **all columns**
 - Check **both diagonals**
- Extension (optional)
 - Modify the function so it works for **any board size N**

Hints

- “Does **any row** contain the same mark in all columns?”

```
for row in self._board:
    if all(cell == mark for cell in row):
        return True
```

- A column is built by fixing the column index and varying the row.

```
for col in range(3):
    if all(self._board[row][col] == mark for row in range(3)):
        return True
```

- Main diagonal, Anti-diagonal, Use index relationships:

Use index relationships:

Main diagonal $\rightarrow i == j$

Anti-diagonal $\rightarrow i + j == N - 1$

Examples Implementation

```
def _is_win(self, mark):
    """Check whether the board configuration is a win for a given player."""
    board = self._board
    N = len(board)

    # check rows
    for r in range(N):
        if all(board[r][c] == mark for c in range(N)):
            return True

    # check columns
    for c in range(N):
        if all(board[r][c] == mark for r in range(N)):
            return True

    # check main diagonal
    if all(board[i][i] == mark for i in range(N)):
        return True

    # check anti-diagonal
    if all(board[i][N - 1 - i] == mark for i in range(N)):
        return True

    return False
```

Outline

- An Example with Search
- Algorithm Analysis
 - What is a Good Algorithm?
 - Pseudo-Code
 - Best-, Worst-, and Average-case Analysis
 - Insertion sort
 - Binary Search
 - Asymptotic Analysis

Finding the Sugared Water

🤔 You have **100 cups of water**, but **only one** contains sugar. How can you find it quickly?

➤ Possible Solution: Naïve Approach (Brute Force)

😞 Taste each cup one by one → **100 checks** needed

➤ **Can We Do Better?**

➤ Is there a **smarter, faster** way?



An Example

- Suppose we use a **list** of integers as the data structure to store a set S of n integers.
- Consider how to support the **search operation**:
 - A function `search( $S$ , search_num)` that returns the index i of `search_num` in the list, or -1 if it is not found.
- Example:
 - $S = \{3, 7, 9, 12, 13, 18, 20, 23, 27\}$
 - Is 20 in S ?
- We give two algorithms to implement it.

An Example: Linear Search

- Simply read the integer of index i for each $i \in [0, n - 1]$.
- If there exists i s.t. $S[i] = \text{search_num}$, return i ;
- Otherwise, return -1.

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

An Example: Binary Search

- Assume sets of integers are sorted.
- Let us compare search_num to the element in the **middle** (the $(n/2)$ -th) of S)

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

searchnum = 18 size = 9

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=5 right=5

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=5

```
searchnum = 28    size = 9
```

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=7 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=7

left=8 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=8

Linear search VS binary search: which one is better?

```
def search(integer_set, search_num):  
    for i in range(len(integer_set)):  
        if integer_set[i] == search_num:  
            return i # Return the index if found  
    return -1 # Return -1 if not found
```

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

Before We Compare: Make a Prediction

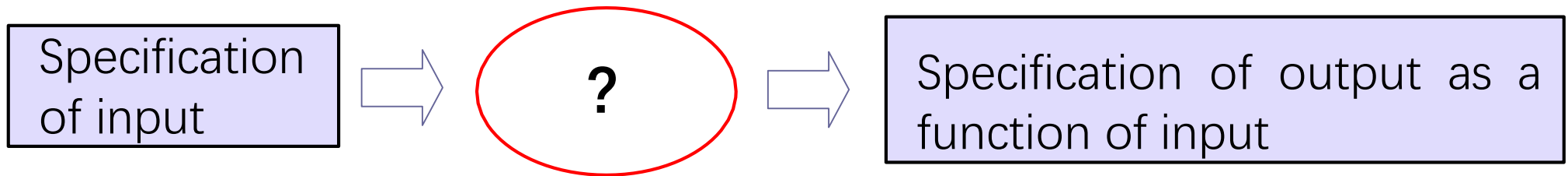
- Suppose:
 - Array size = **1,000**
 - Target is **not in the array**
- **Question:**
 - How many comparisons in **linear search**?
 - How many comparisons in **binary search**?

Outline

- An Example with Search
- **Algorithm Analysis**
 - What is a Good Algorithm?
 - Pseudo-Code
 - Best-, Worst-, and Average-case Analysis
 - Asymptotic Analysis

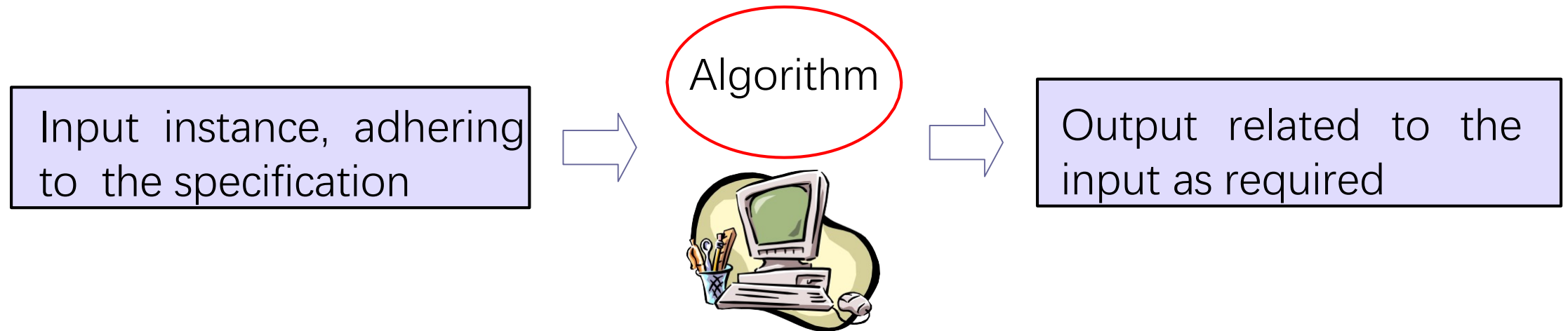
Algorithmic problem

- Infinite number of input instances satisfying the specification.
 - E.g. A sorted, non-decreasing sequence of natural numbers of non-zero, finite length:
 - 1, 20, 908, 909, 100000, 1000000000.
 - 3.



Algorithmic Solution

- Algorithm describes actions on the input instance
- Many correct algorithms for the same algorithmic problem



What is a Good Algorithm?

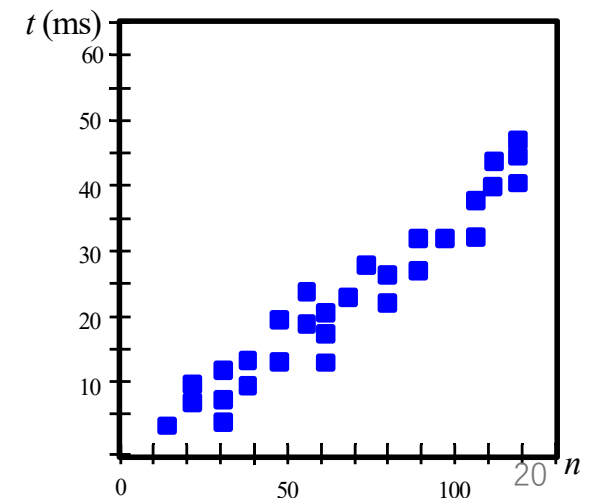
- Efficient:
 - Running time
 - Space used
- Efficiency as a function of input size:
 - The number of bits in an input number
 - Number of data elements (numbers, points)

Measuring the Running Time

How should we measure the running time of an **algorithm**?

Experimental Study

- Write a **program** that implements the algorithm
- Run the program with data sets of varying size and composition.
- Use a system call to get an accurate measure of the actual running time.



Limitations of Experimental Studies

- It is necessary to **implement** and test the algorithm in order to determine its running time.
- Experiments done only on a **limited set of inputs**,
 - may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same **hardware and software environments** needed

Beyond Experimental Studies

We will develop a **general methodology** for analyzing running time of algorithms. This approach

- Uses a **high-level description** of the algorithm instead of testing one of its implementations.
- Takes into account **all possible inputs**.
- Allows one to evaluate the efficiency of any algorithm in a way that is **independent of the hardware and software environment**.

Example

- **Algorithm arrayMax(A, n):**
 - Input: An array A storing n integers. Output: The maximum element in A.

Pseudo-Code (imperative)

- A mixture of natural language and high-level programming concepts that describes the main ideas behind a generic implementation of a data structure or algorithm.

- E.g. **algorithm arrayMax(A, n):**

Input: An array A storing n integers.

Output: The maximum element in A.

currentMax $\leftarrow$ A[0]

for i $\leftarrow$ 1 to n-1 do

 if currentMax < A[i] then currentMax $\leftarrow$ A[i]

return currentMax

Pseudo-Code

It is more structured than usual prose but less formal than a programming language

- Expressions:
 - use standard mathematical symbols to describe numeric and boolean expressions
 - use $\leftarrow$ for assignment (“=” in Java)
 - use = for equality relationship (“==” in Java)
- Method Declarations:
 - **algorithm** name(param1, param2)

Pseudo Code

There are several styles commonly used.

- Programming Constructs:
 - decision structures: **if ... then ... [else ...]**
 - while-loops: **while ... do**
 - repeat-loops: **repeat ... until ...**
 - for-loop: **for ... do**
 - array indexing: **A[i], A[i,j]**
- Methods:
 - calls: object method(args)
 - returns: **return** value

Algorithmic Problem: Find the Second Largest Element

- **Input:**
An array A of n integers, where $n \geq 2$
- **Output:**
The **second largest** element in the array
- **Constraints:**
 - You are **not allowed to sort** the array
 - The algorithm should work for **any valid input**

- **Example:**

Input: [3, 10, 5, 8, 2]

Output: 8

Analysis of Algorithms

- **Primitive Operation:** Low-level operation independent of programming language.
- Can be identified in pseudo-code. For eg:
 - Data movement (assign)
 - Control (branch, subroutine call, return)
 - arithmetic and logical operations (e.g. addition, comparison)
- By inspecting the pseudo-code, we can count the number of primitive operations executed by an algorithm.

Sorting a Hand of Cards

🤔 You have a **hand of playing cards** in **random order**.

- Q1: How can you arrange them in **ascending order** efficiently?
- Q2: When giving a sequence of new cards one by one, how should they **be inserted** in the right position?



Another Example: Sorting

INPUT

sequence of numbers

$a_1, a_2, a_3, \dots, a_n$

2 5 4 10 7



OUTPUT

a permutation of the
sequence of numbers

$b_1, b_2, b_3, \dots, b_n$

2 4 5 7 10

Correctness (requirements for the output)

For any given input the algorithm halts with the output:

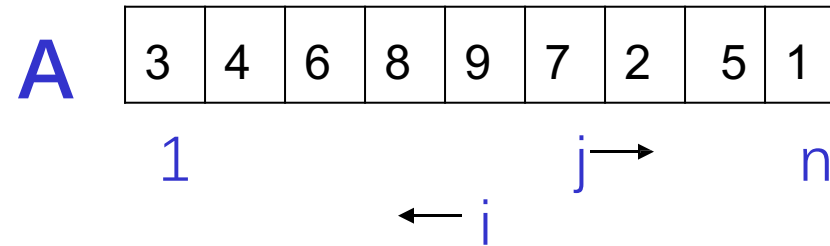
- $b_1 < b_2 < b_3 < \dots < b_n$
- $b_1, b_2, b_3, \dots, b_n$ is a permutation of $a_1, a_2, a_3, \dots, a_n$

Running time

Depends on

- number of elements (n)
- how (partially) sorted they are
- algorithm

Insertion Sort

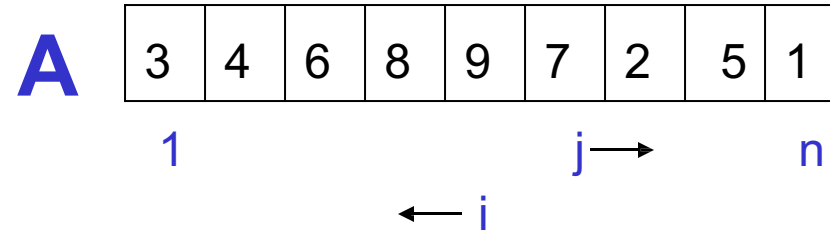


Strategy

- Start “empty handed”
- Insert a card in the right position of the already sorted hand
- Continue until all cards are inserted/sorted

INPUT: $A[0..n-1]$ – an array of integers
OUTPUT: a permutation of A such that $A[0] \leq A[1] \leq \dots \leq A[n-1]$

Insertion Sort



Strategy

- Start “empty handed”
- Insert a card in the right position of the already sorted hand
- Continue until all cards are inserted/sorted

INPUT: $A[0..n-1]$ – an array of integers
OUTPUT: a permutation of A such that
 $A[0] \leq A[1] \leq \dots \leq A[n-1]$

```
for j ← 1 to n-1 do
    key ← A[j]
    //insert A[j] into the sorted sequence
    A[0..j-1]
    i ← j-1
    while i ≥ 0 and A[i] > key
        do A[i+1] ← A[i] i--
    A[i+1] ← key
```

Analysis of Insertion Sort

```

for j ← 1 to n-1 do
  key ← A[j]
  //insert A[j] into the sorted
  sequence A[0..j-1]
  i ← j - 1
  while i ≥ 0 and A[i] > key
    do A[i+1] ← A[i]
        i--
  A[i+1] ← key
    
```

cost	Times
c_1	n
c_2	$n-1$
0	$n-1$
c_3	$n-1$
c_4	$\sum_{j=1}^{n-1} t_j$
c_5	$\sum_{j=1}^{n-1} (t_j - 1)$
c_6	$\sum_{j=1}^{n-1} (t_j - 1)$
c_7	$n-1$

t_j : the number of while-loops used for inserting $A[j]$

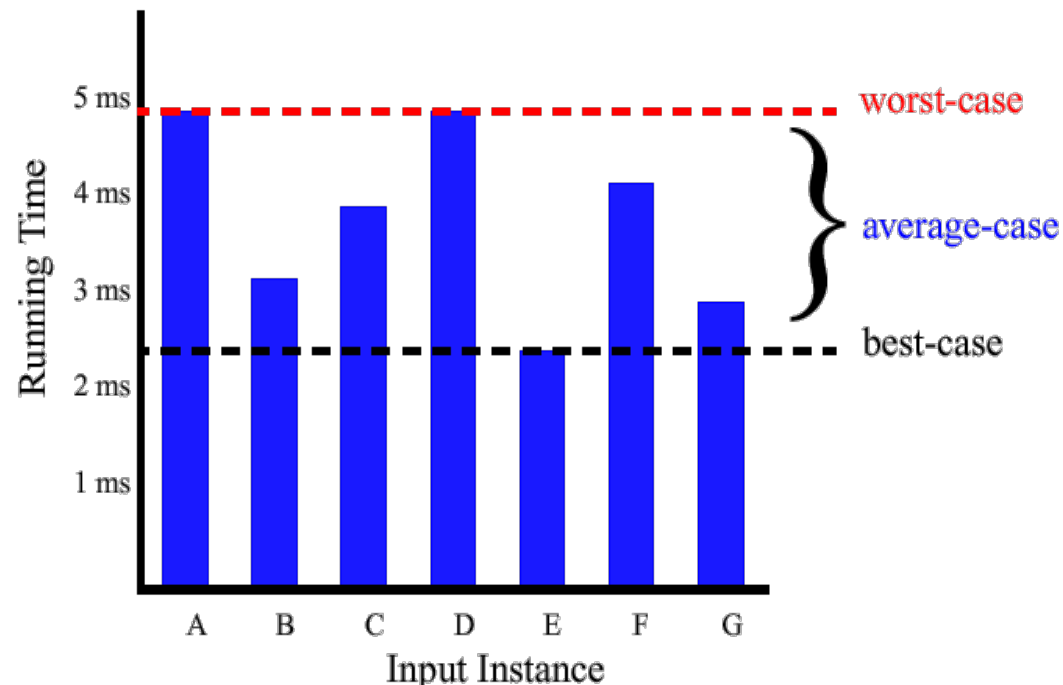
$$\begin{aligned}
 \text{Total time} = & n \cdot (c_1 + c_2 + c_3 + c_7) + \sum_{j=1}^{n-1} t_j \cdot (c_4 + c_5 + c_6) \\
 & - (c_2 + c_3 + c_7) - (c_5 + c_6) \cdot (n - 1)
 \end{aligned}$$

Best/Worst/Average Case (1)

- Total time = $n \cdot (c_1 + c_2 + c_3 + c_7) + \sum_{j=1}^{n-1} t_j \cdot (c_4 + c_5 + c_6) - (c_2 + c_3 + c_7) - (c_5 + c_6) \cdot (n - 1)$
- **Best case:**
 - elements already sorted; $t_j = 1$; running time is at the order of n
- **Worst case:**
 - elements are sorted in inverse order; $t_j = j + 1$; running time is at the order of n^2
- **Average case:**
 - $t_j = (j + 1)/2$; running time is at the order of n^2

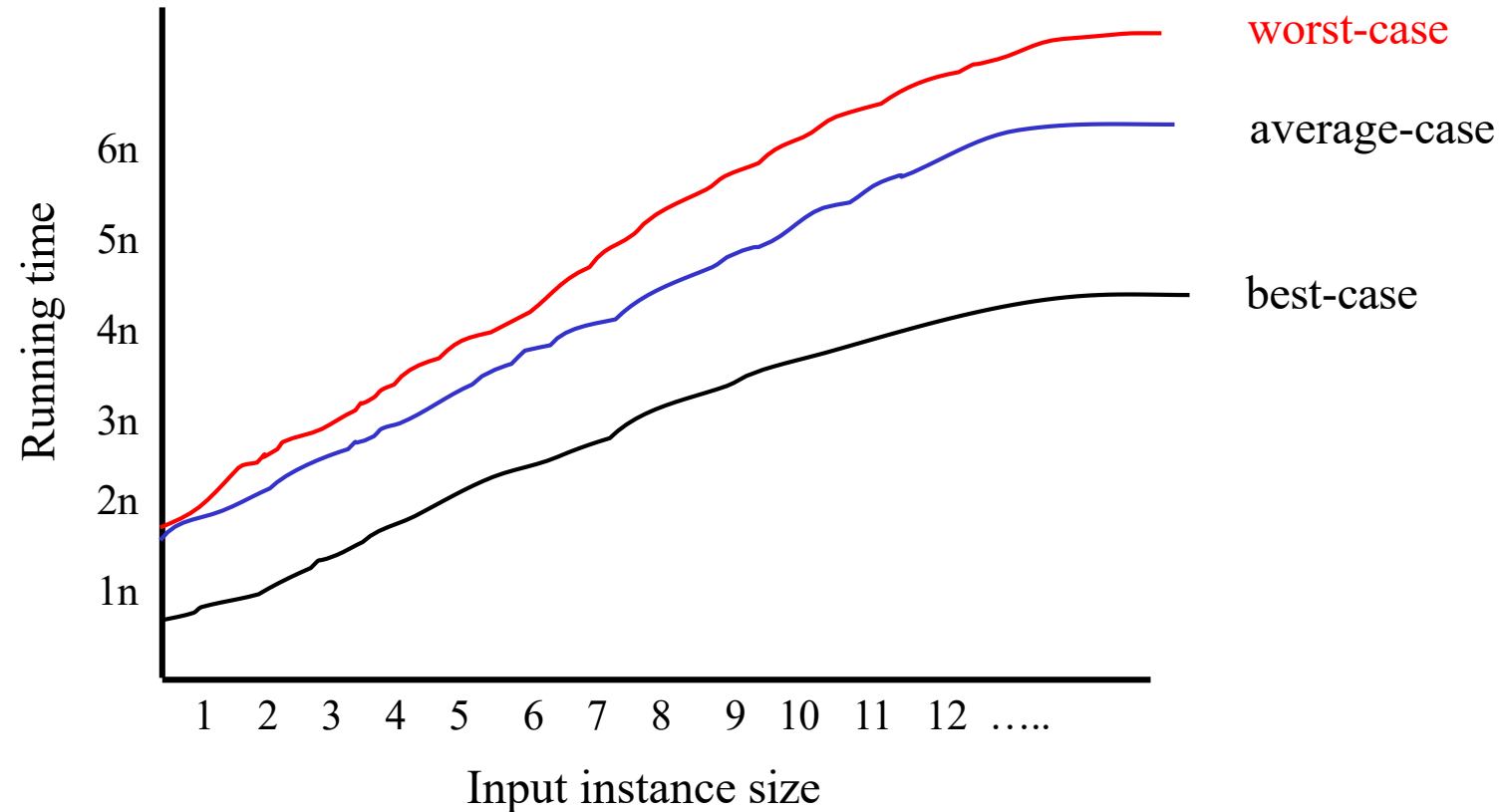
Best/Worst/Average Case (2)

- For a specific size of input n , investigate running times for different input instances:
- The **worst-case time** of an algorithm under a problem size n is the **largest** running time of the algorithm on all (possibly an infinite number of) the size- n inputs.



Best/Worst/Average Case (3)

- For inputs of all sizes:



Best/Worst/Average Case (4)

- **Worst case** is usually used: It is an upper- bound and in certain application domains (e.g., air traffic control, surgery) knowing the **worst-case** time complexity is of crucial importance
- For some algos **worst case** occurs fairly often
- **Average case** is often as bad as **worst case**
- Finding **average case** can be very difficult

Worst-Case Time of Binary Search

- Let us call the integers whose indices fall within [left, right] as active elements.
- Refer to each while-loop as an iteration. Each iteration performs a constant c number of atomic operations. A loose upper bound is $c \leq 10$.
- How many iterations are there?

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

Worst-Case Time of Binary Search

- How many iterations are there?
- **Property:** After the i -th iteration, the number of active elements is at most $n/2^i$.
- Suppose there are h iterations in total.
 - It holds that $\frac{n}{2^h} \geq 1$ and thus $h \leq 1 + \log_2 n$
- The worst-case time of binary search is thus at most
 - $10 \cdot (1 + \log_2 n)$
- The worst-case time of linear search is at most
 - $4 \cdot (1 + n)$ (try verifying this yourself)
- When n is large, binary search is much faster than linear search

Cost Analysis Exercise 1: Linear Search

```
algorithm linearSearch(A, n, x):  
    for i ← 0 to n - 1 do  
        if A[i] = x then  
            return i  
    return -1
```

- What are the Best/Average/Worse Cost?

Case	Cost
Best	1 comparison
Worst	n comparisons
Average	$(n + 1) / 2$ comparisons

Cost Analysis Exercise 2: Conditional Nested Loop

```
algorithm mystery(A, n):  
    count ← 0  
    for i ← 0 to n - 1 do  
        if A[i] > 0 then  
            for j ← 0 to n - 1 do  
                count ← count + 1  
    return count
```

Best/Worst cost?

What input causes each?

Case	Cost
Best	(all $A[i] \leq 0$) $\Theta(n)$
Worst	(all $A[i] > 0$) $\Theta(n^2)$
Average	Depends on distribution

Cost Analysis Exercise 3: Prefix Averages

```
for i ← 0 to n - 1 do
    sum ← 0
    for j ← 0 to i do
        sum ← sum + X[j]
    Avg[i] = sum / (1+i)
```

Best/Worst cost

Best/worst cost: $\sum_{i=0}^{n-1} (i+1) = \frac{n(n+1)}{2} = \Theta(n^2)$

Asymptotic Analysis

- Consider an arbitrary problem. Suppose that Algorithm 1 runs in
 - $1000n \cdot c_{multi} + 10n \cdot c_{mem}$ time,
- where c_{multi} is the time of one multiplication and c_{mem} the time of one memory access; Algorithm 2 runs in
 - $10n \cdot c_{multi} + 100n \cdot c_{mem}$ time.
- Which one is better? **It depends** on the specific values of c_{multi} and c_{mem} , which vary from machine to machine.

Why Not Constants?

- Consider an arbitrary problem. Suppose that Algorithm 1 runs in
 - $1000n \cdot c_{multi} + 10n \cdot c_{mem}$ time,
- where c_{multi} is the time of one multiplication and c_{mem} the time of one memory access; Algorithm 2 runs in
 - $10n \cdot c_{multi} + 100n \cdot c_{mem}$ time.
- In mathematics, we want to make a universally correct conclusion, which holds on all machines:
 - Regardless of the machine, their costs differ by at most a constant factor.

Asymptotic Analysis

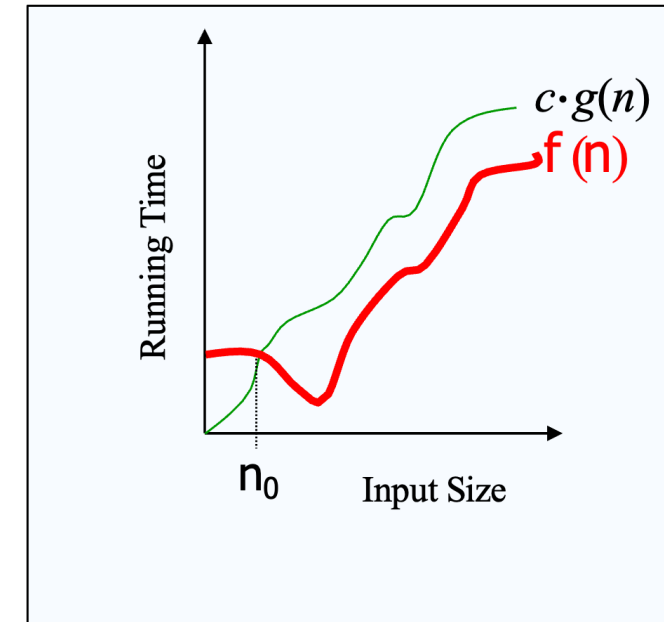
- **Goal:** to simplify analysis of running time by getting rid of “details”, which may be affected by specific implementation and hardware
 - like “rounding”: $1,000,001 \approx 1,000,000$
 - $3n^2 \approx n^2$
- Capturing the essence:
 - The **growth** of the running time with the problem size n
 - We care about the efficiency of the problem when n is **large** (think: why?)

Asymptotic Analysis

- Capturing the essence:
 - The **growth** of the running time with the problem size n
 - We care about the efficiency of the problem when n is **large** (think: why?)
- **Example:** suppose that
 - Algorithm 1 requires n atomic operations;
 - While Algorithm 2 requires $10000 \cdot \log_2 n$ operations.
 - For $n = 2^{30}$ (roughly 10^9), Algorithm 2 is faster by a factor of $\frac{n}{10000 \cdot \log_2 n} > 3579$
 - Algorithm 2 is considered better than Algorithm 1 in theoretical computer science.

Asymptotic Notation

- The “big-Oh” O-Notation
 - asymptotic upper bound
 - let $f(n)$ and $g(n)$ be two functions of n .
 - we say that $f(n)$ grows asymptotically no faster than $g(n)$ if there exists positive constants c and n_0 , s.t.
$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0.$$
 - we can denote this by $f(n) = O(g(n))$.



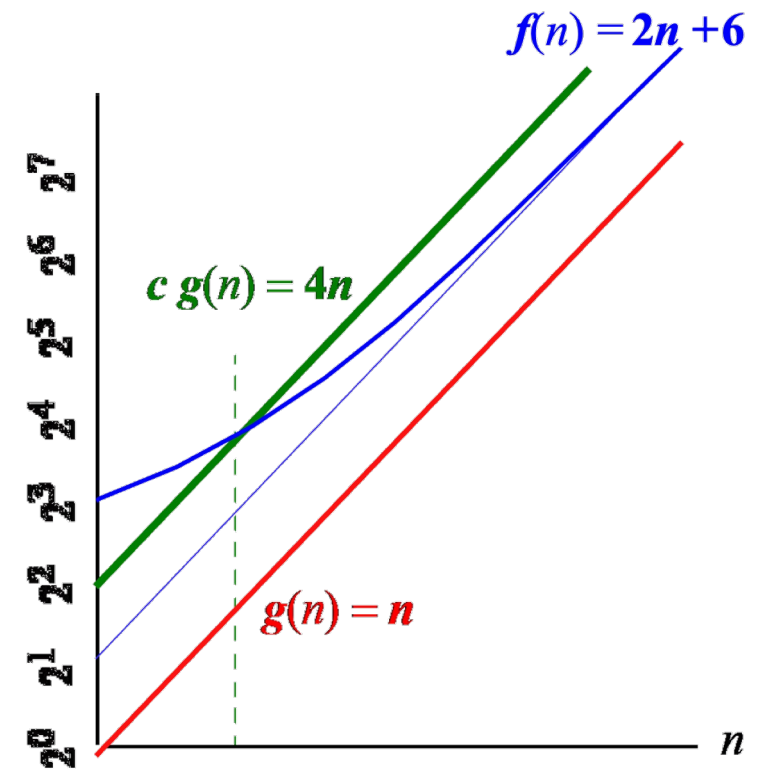
- Used for worst-case analysis
- Art of Computer Science:
 - Minimize the growth of running time in solving a problem

Example

For functions $f(n)$ and $g(n)$ there are positive constants c and n_0 such that: $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Conclusion:

$$2n + 6 = O(n).$$



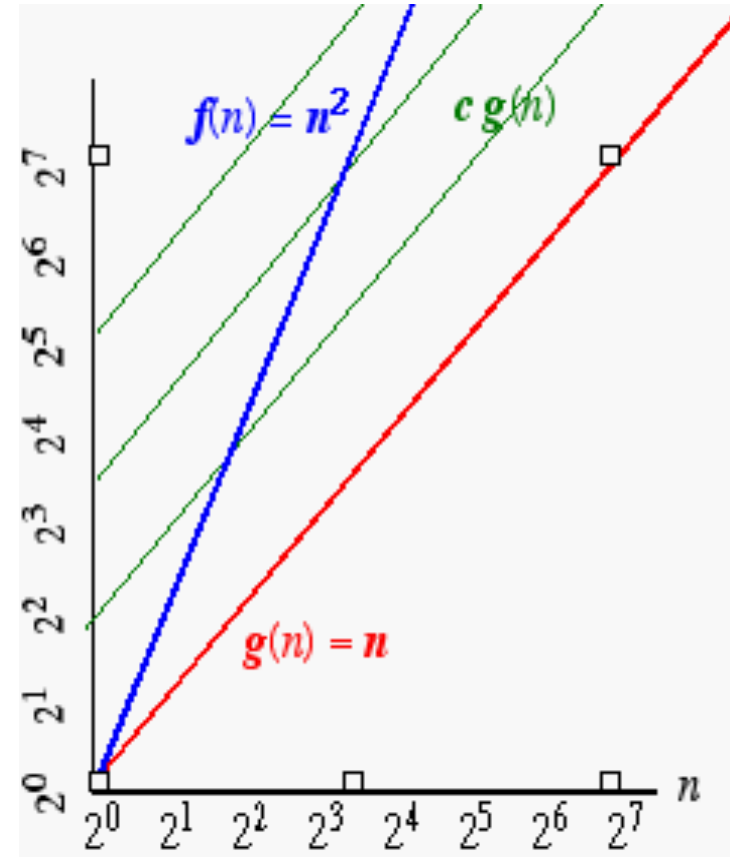
Another Example

On the other hand...

n^2 is not $O(n)$ because there is no c and n_0 such that:

$$n^2 \leq cn \text{ for } n \geq n_0$$

The graph to the right illustrates that no matter how large a c is chosen there is an n big enough that $n^2 > cn$.



Example

Verify the following:

$$\begin{aligned}100000000 &= O(1) \\100\sqrt{n} + 10n &= O(n) \\1000n^{1.5} &= O(n^2) \\(\log_2 n)^3 &= O(\sqrt{n}) \\(\log_2 n)^{999999} &= O(n^{0.0000001})\end{aligned}$$

All True

Asymptotic Notation

- **Simple Rule:** drop lower order terms and constant factors.
 - $50n \log n$ is $O(n \log n)$
 - $7n - 3$ is $O(n)$
 - $8n^2 \log n + 5n^2 + n$ is $O(n^2 \log n)$
- **An interesting fact:**
 $\log_{b_1} n = O(\log_{b_2} n)$ for any constants $b_1 > 1$ and $b_2 > 1$.
- Because of the above, in computer science, we omit all the constant logarithm bases in big-O. For example, instead of $O(\log_2 n)$, we will simply write $O(\log n)$.

Asymptotic Analysis of Running Time

- Use O -notation to express number of primitive operations executed as function of input size.
- Comparing asymptotic running times
 - an algorithm that runs in $O(n)$ time is better than one that runs in $O(n^2)$ time
 - similarly, $O(\log n)$ is better than $O(n)$
 - hierarchy of functions: $\log n < n < n^2 < n^3 < 2^n$
- **Caution!** Beware of very large constant factors. An algorithm running in time $1,000,000 n$ is still $O(n)$ but might be less efficient than one running in time $2n^2$, which is $O(n^2)$

Example of Asymptotic Analysis

Algorithm prefixAverages1(X):

Input: An n -element array X of numbers.

Output: An n -element array A of numbers such that $A[i]$ is the average of elements $X[0], \dots, X[i]$.

for $i \leftarrow 0$ **to** $n-1$ **do**

$a \leftarrow 0$

for $j \leftarrow 0$ **to** i **do**

$a \leftarrow a + X[j]$ ← 1 step

$A[i] \leftarrow a/(i+1)$

return array A

i iterations with
 $i=0,1,2,\dots,n-1$

n iterations

Analysis: running time is $O(n^2)$

A Better Algorithm

Algorithm prefixAverages2(X):

Input: An n -element array X of numbers.

Output: An n -element array A of numbers such that $A[i]$ is the average of elements $X[0], \dots, X[i]$.

$s \leftarrow 0$

for $i \leftarrow 0$ **to** n **do**

$s \leftarrow s + X[i]$

$A[i] \leftarrow s/(i+1)$

return array A

Analysis: Running time is $O(n)$

Asymptotic Notation (terminology)

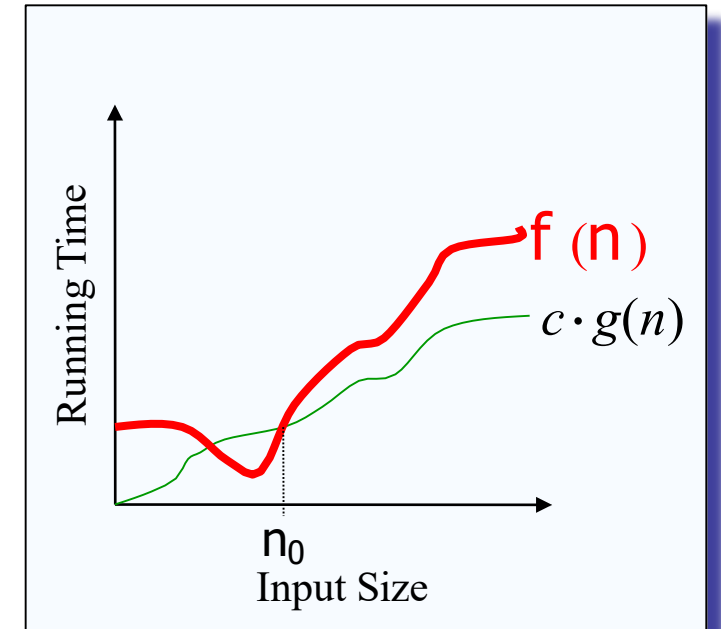
- Special classes of algorithms:
 - Logarithmic: $O(\log n)$
 - Linear: $O(n)$
 - Quadratic: $O(n^2)$
 - Polynomial: $O(n^k), k \geq 1$
 - Exponential: $O(a^n), a > 1$
- “Relatives” of the Big-Oh
 - $\Omega(f(n))$: Big Omega -asymptotic lower bound
 - $\Theta(f(n))$: Big Theta -asymptotic tight bound

Asymptotic Notation

- The “big-Omega” Ω –Notation
 - Asymptotic lower bound
 - Let $f(n)$ and $g(n)$ be two functions of n .
 - We say that $f(n)$ grows asymptotically no slower than $g(n)$ if there exists positive constants c and n_0 , s.t.

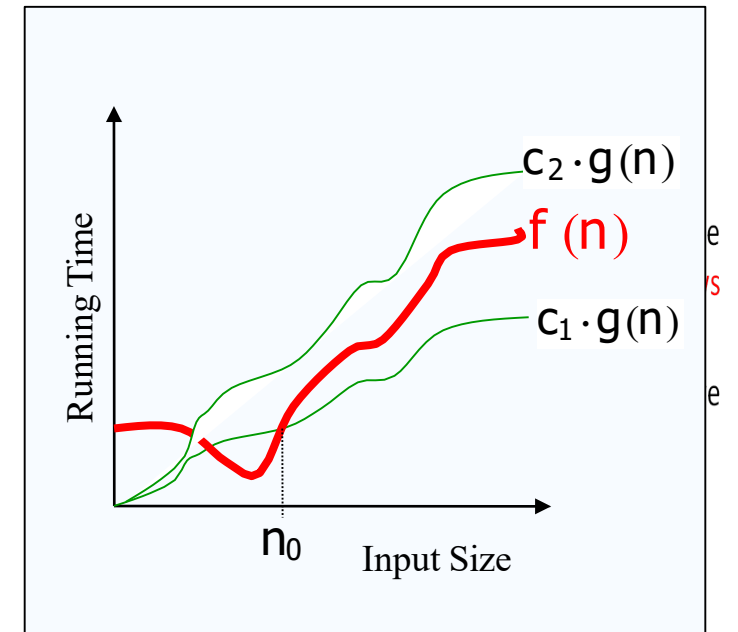
$$f(n) \geq c \cdot g(n) \text{ for all } n \geq n_0.$$

- We can denote this by $f(n) = \Omega(g(n))$.
- Used to describe best-case running times or lower bounds for algorithmic problems
- E.g., lower-bound for searching in an unsorted array is $\Omega(n)$.



Asymptotic Notation

- The “big-Theta” Θ –Notation
 - asymptotically tight bound
 - let $f(n)$ and $g(n)$ be two functions of n .
 - If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then we define $f(n) = \Theta(g(n))$ to indicate that $f(n)$ grows asymptotically as fast as $g(n)$.
 - $f(n) = \Theta(g(n))$ if and only if there exists positive constants c_1 , c_2 and n_0 , s.t.
$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0.$$



Example

Verify the following:

$$\log_2 n = \Omega(1)$$
$$n^{0.000001} = \Omega(\log_2 n)^{999999})$$

All True

$$\frac{2^n}{1000} = \Omega(n^{999999})$$

$$1000 + 30 \log_2 n + 1.5\sqrt{n} = \Theta(\sqrt{n})$$

$$n^2 + 2n + 1 = \Theta(n^2)$$

Example

Verify the following:

$$\begin{array}{ll} n^2 = O(n \log n) & \text{False} \\ n + \log n = \Theta(\log n) & \text{False} \\ n \log n = O(n^2) & \text{True} \end{array}$$

Asymptotic Notation

Two more asymptotic notations

- “Little-Oh” notation $f(n)$ is $o(g(n))$ non-tight analogue of Big-Oh
 - For every $c > 0$, there should exist n_0 , s.t. $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
 - Used for **comparisons** of running times. If $f(n)$ is $o(g(n))$, it is said that $g(n)$ dominates $f(n)$.
- “Little-omega” notation $f(n)$ is $\omega(g(n))$ non-tight analogue of Big-Omega

Asymptotic Notation

- Analogy with real numbers

• $f(n)$ is $O(g(n))$	$\cong$	$f \leq g$
• $f(n)$ is $\Omega(g(n))$	$\cong$	$f \geq g$
• $f(n)$ is $\Theta(g(n))$	$\cong$	$f = g$
• $f(n)$ is $o(g(n))$	$\cong$	$f < g$
• $f(n)$ is $\omega(g(n))$	$\cong$	$f > g$

- Abuse of notation: $f(n) = O(g(n))$ actually means $f(n) = \Theta(g(n))$

Comparison of Running Times

Running Time	Maximum problem size (n)		
	1 second	1 minute	1 hour
$400n$	2500	150000	9000000
$20n \log n$	4096	166666	7826087
$2n^2$	707	5477	42426
n^4	31	88	244
2^n	19	25	31